

Worked Numerical Example

Take a tiny toy LCG: $m = 16$, $a = 5$, $c = 3$, $X_0 = 7$.

LCG

$$X_{n+1} = (5X_n + 3) \bmod 16$$

	X_0	X_1	X_2	X_3	X_4	X_5	$X_6 \dots X_{16}$	X_n
n	0	1	2	3	4	5		
X_n	7	6	1	8	11	10		
$U_n = X_n/16$	0.4375	0.375	0.0625	0.500	0.6875	0.625		

$1, 3, 11, 13, 15$
 $16 = 2 \times 2 \times 2 \times 2$
 $\frac{X_n}{m} = U_n$

- Check Hull–Dobell: $\gcd(3, 16) = 1$ ✓; prime factor of 16 is 2, and $a - 1 = 4$ divisible by 2 ✓; m divisible by 4 and $a - 1 = 4$ divisible by 4 ✓ $\Rightarrow$ full period 16.

Worked Numerical Example

Take a tiny toy LCG: $m = 16$, $a = 5$, $c = 3$, $X_0 = 7$.

$$X_{n+1} = (5X_n + 3) \bmod 16$$

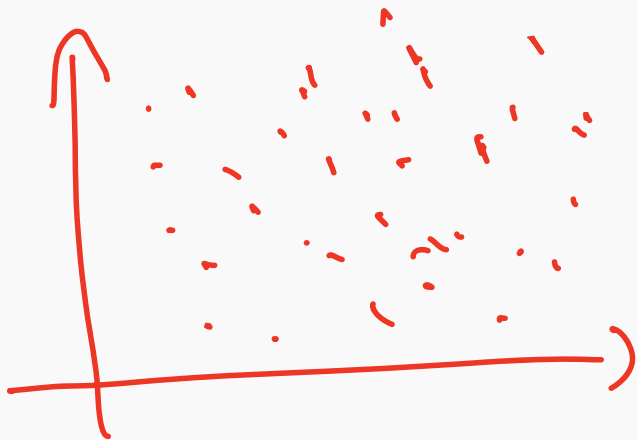
n	0	1	2	3	4	5
X_n	7	6	1	8	11	10
$U_n = X_n/16$	0.4375	0.375	0.0625	0.500	0.6875	0.625

- Check Hull–Dobell: $\gcd(3, 16) = 1$ ✓; prime factor of 16 is 2, and $a - 1 = 4$ divisible by 2 ✓; m divisible by 4 and $a - 1 = 4$ divisible by 4 ✓ $\Rightarrow$ full period 16.
- In practice m is astronomically large (e.g. $2^{31} - 1$ or bigger), so the period is enormous.

The Lattice Structure Problem

Marsaglia's Theorem (1968) – “Successive random numbers generated by an LCG do not fill space uniformly.”

Overlapping k -tuples $(U_n, U_{n+1}, \dots, U_{n+k-1})$ generated by an LCG do *not* fill the k -dimensional unit cube uniformly – they lie on a limited number of parallel hyperplanes. **We will see this in Lab**

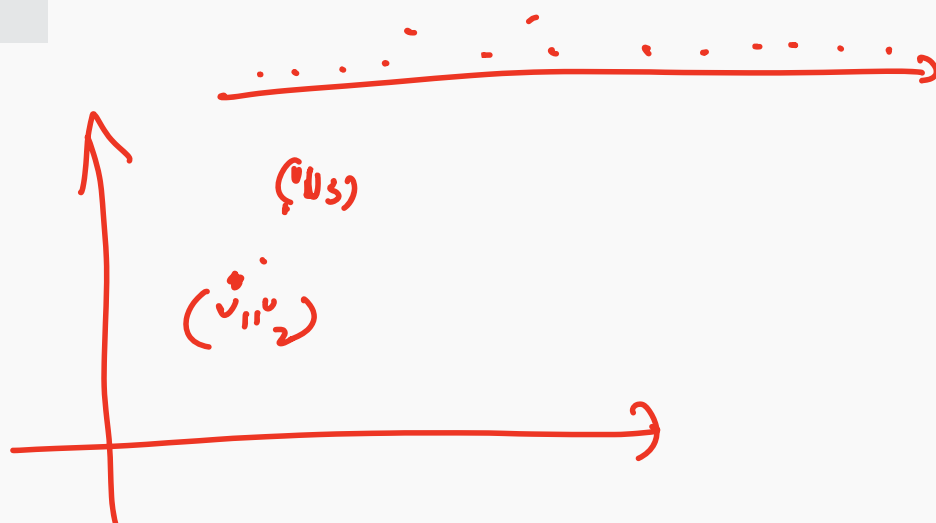


$$X_n = (aX_{n-1} + c) \bmod m$$

$$U_n = \frac{X_n}{m}$$

$$\hookrightarrow U_1, U_2, \dots \in [0, 1]$$

$$(U_1, U_2), (U_2, U_3), (U_3, U_4), \dots \in [0, 1]^2$$



The Lattice Structure Problem

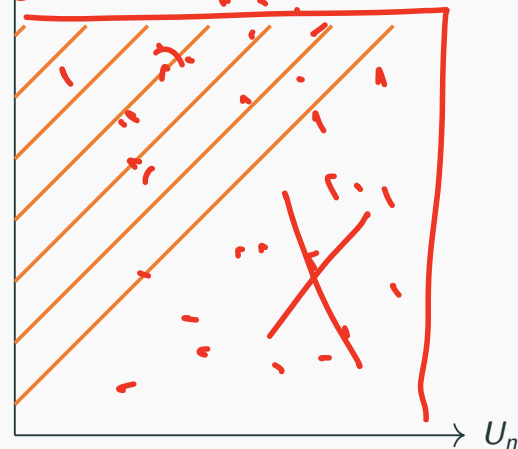
Marsaglia's Theorem (1968) – “Successive random numbers generated by an LCG do not fill space uniformly.”

Overlapping k -tuples $(U_n, U_{n+1}, \dots, U_{n+k-1})$ generated by an LCG do *not* fill the k -dimensional unit cube uniformly – they lie on a limited number of parallel hyperplanes. **We will see this in Lab**

- This is a structural flaw of *all* LCGs.
- It can bias simulations that rely on multivariate uniform draws (e.g., Monte Carlo integration in several dimensions).

$$X_{n+1} = (aX_n + c) \mid m$$

$$U_{n+1} = \frac{X_{n+1}}{m} \quad \dots \quad (0,1)^2$$



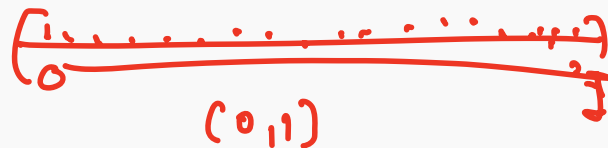
$$\iint f(x,y) dx dy$$

$$\int \dots \int f(x,y) dx \dots dy$$

Points cluster on parallel lines/planes

$$U_1, U_2, \dots, U_n, \dots$$

$$(U_1, U_{100}) (U_2, U_{101}) \dots$$



Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937)**: default generator in R, Python, MATLAB, NumPy. (LCG)
 - Astronomically long period $2^{19937} - 1$. $= m$ ✓ $\Rightarrow y_{n+1} = (ay_n + c) \mid m$
 - Passes almost all standard statistical test suites. $\checkmark$
 - Unlike LCGn, which relies on just the previous number (X_{-n}), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ Patterns almost disappear ✓
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation. ✓

Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937)**: default generator in R, Python, MATLAB, NumPy.
 - Astronomically long period $2^{19937} - 1$.
 - Passes almost all standard statistical test suites.
 - Unlike LCGn, which relies on just the previous number ($X - n$), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ Patterns almost disappear
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation.
- **Xorshift / xoshiro family**: extremely fast, use bitwise XOR and shift operations; good statistical quality with tiny state.

$\Rightarrow$ 110100 $\hookrightarrow$
10101 $\hookrightarrow$
10101 $\hookrightarrow$

whole
course

$\hookrightarrow U(0,1)$

Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937)**: default generator in R, Python, MATLAB, NumPy.
 - Astronomically long period $2^{19937} - 1$.
 - Passes almost all standard statistical test suites.
 - Unlike LCGn, which relies on just the previous number (X_{n-1}), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ **Patterns almost disappear**
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation.
- **Xorshift / xoshiro family**: extremely fast, use bitwise XOR and shift operations; good statistical quality with tiny state.
- **PCG (Permuted Congruential Generator)**: combines an LCG core with a permutation output function – small state, excellent statistical properties.

$x_1, x_2, x_3, \dots, x_5$
 $x_3, x_{100}, x_{50}, \dots$
 $(x_3, x_{100}) (x_{100}, x_{50}) \rightarrow$

Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937)**: default generator in R, Python, MATLAB, NumPy.
 - Astronomically long period $2^{19937} - 1$.
 - Passes almost all standard statistical test suites.
 - Unlike LCGn, which relies on just the previous number ($X - n$), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ **Patterns almost disappear**
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation.
- **Xorshift / xoshiro family**: extremely fast, use bitwise XOR and shift operations; good statistical quality with tiny state.
- **PCG (Permuted Congruential Generator)**: combines an LCG core with a permutation output function – small state, excellent statistical properties.
- **Cryptographic RNGs** (e.g., ChaCha20, hardware TRNGs): needed when unpredictability itself matters (security, gambling), not just statistical quality.

$$X_{n+1} = (aX_n + c) \bmod m$$


Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937):** default generator in R, Python, MATLAB, NumPy.
 - Astronomically long period $2^{19937} - 1$.
 - Passes almost all standard statistical test suites.
 - Unlike LCGn, which relies on just the previous number ($X - n$), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ **Patterns almost disappear**
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation.
- **Xorshift / xoshiro family:** extremely fast, use bitwise XOR and shift operations; good statistical quality with tiny state.
- **PCG (Permuted Congruential Generator):** combines an LCG core with a permutation output function – small state, excellent statistical properties.
- **Cryptographic RNGs** (e.g., ChaCha20, hardware TRNGs): needed when unpredictability itself matters (security, gambling), not just statistical quality.

Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937)**: default generator in R, Python, MATLAB, NumPy.
 - Astronomically long period $2^{19937} - 1$.
 - Passes almost all standard statistical test suites.
 - Unlike LCGn, which relies on just the previous number ($X - n$), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ **Patterns almost disappear**
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation.
- **Xorshift / xoshiro family**: extremely fast, use bitwise XOR and shift operations; good statistical quality with tiny state.
- **PCG (Permuted Congruential Generator)**: combines an LCG core with a permutation output function – small state, excellent statistical properties.
- **Cryptographic RNGs** (e.g., ChaCha20, hardware TRNGs): needed when unpredictability itself matters (security, gambling), not just statistical quality.

Note: For this course, think of a modern PRNG (e.g. Mersenne Twister) as a “black box” that reliably delivers a stream $U_1, U_2, U_3, \dots$ $\overset{\text{approx}}{\sim}$ i.i.d. $\text{Unif}(0, 1)$. Everything else in this unit is about *transforming* that stream into other distributions.

Outline

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

From PRNG Output to $U(0, 1)$

Strategies for Non-Uniform Generation

Inverse Transform Method

Acceptance–Rejection Method

Box–Muller Transformation

Ratio-of-Uniforms Method

Diagnosing Randomness

Simulation Error

Unit Summary

The Building Block: Uniform(0, 1)

$$U_1, \dots, U_n \sim [0, 1]$$

- A PRNG's raw output is an integer $X_n \in \{0, 1, \dots, m-1\}$.

$$\int f(x) dx$$

↑

The Building Block: Uniform(0, 1)

- A PRNG's raw output is an integer $X_n \in \{0, 1, \dots, m-1\}$.
- Dividing by m (or $m-1$, depending on convention) rescales it to $(0, 1)$: $U_n = X_n/m$.

The Building Block: Uniform(0, 1)

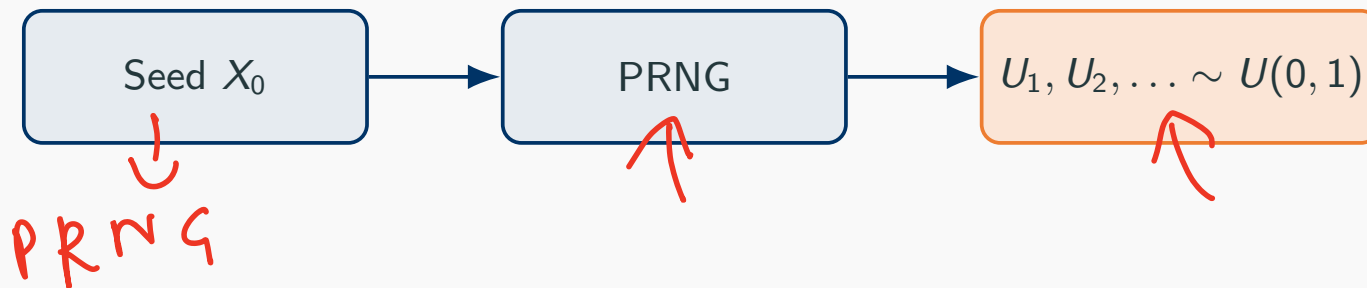
- A PRNG's raw output is an integer $X_n \in \{0, 1, \dots, m - 1\}$.
- Dividing by m (or $m - 1$, depending on convention) rescales it to $(0, 1)$: $U_n = X_n/m$.
- This stream of (approximately) i.i.d. $U(0, 1)$ variables is the **universal raw material** from which every other distribution in this course is built.

The Building Block: $\text{Uniform}(0, 1)$

- A PRNG's raw output is an integer $X_n \in \{0, 1, \dots, m - 1\}$.
- Dividing by m (or $m - 1$, depending on convention) rescales it to $(0, 1)$: $U_n = X_n/m$.
- This stream of (approximately) i.i.d. $U(0, 1)$ variables is the **universal raw material** from which every other distribution in this course is built.

The Building Block: $\text{Uniform}(0, 1)$

- A PRNG's raw output is an integer $X_n \in \{0, 1, \dots, m - 1\}$.
- Dividing by m (or $m - 1$, depending on convention) rescales it to $(0, 1)$: $U_n = X_n/m$.
- This stream of (approximately) i.i.d. $U(0, 1)$ variables is the **universal raw material** from which every other distribution in this course is built.



From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. Inverse Transform Method – invert the target CDF directly: $X = F^{-1}(U)$.

From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. **Inverse Transform Method** – invert the target CDF directly: $X = F^{-1}(U)$.
2. **Transformation Methods** – express X as a deterministic function of one or more uniforms/other random variables with known distributions (e.g. Box–Muller for the Normal).

From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. **Inverse Transform Method** – invert the target CDF directly: $X = F^{-1}(U)$.
2. **Transformation Methods** – express X as a deterministic function of one or more uniforms/other random variables with known distributions (e.g. Box–Muller for the Normal).
3. **Acceptance–Rejection Method** – sample from an easy “envelope” distribution and probabilistically accept/reject to correct the shape.

From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. **Inverse Transform Method** – invert the target CDF directly: $X = F^{-1}(U)$.
2. **Transformation Methods** – express X as a deterministic function of one or more uniforms/other random variables with known distributions (e.g. Box–Muller for the Normal).
3. **Acceptance–Rejection Method** – sample from an easy “envelope” distribution and probabilistically accept/reject to correct the shape.
4. **Ratio-of-Uniforms Method** – generate a pair of uniforms over a cleverly chosen 2-D region whose ratio has the target distribution.

From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. **Inverse Transform Method** – invert the target CDF directly: $X = F^{-1}(U)$.
2. **Transformation Methods** – express X as a deterministic function of one or more uniforms/other random variables with known distributions (e.g. Box–Muller for the Normal).
3. **Acceptance–Rejection Method** – sample from an easy “envelope” distribution and probabilistically accept/reject to correct the shape.
4. **Ratio-of-Uniforms Method** – generate a pair of uniforms over a cleverly chosen 2-D region whose ratio has the target distribution.

From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. **Inverse Transform Method** – invert the target CDF directly: $X = F^{-1}(U)$.
2. **Transformation Methods** – express X as a deterministic function of one or more uniforms/other random variables with known distributions (e.g. Box–Muller for the Normal).
 $\rightarrow \Phi(x) = \int_{-\infty}^x \phi(t) dt$
3. **Acceptance–Rejection Method** – sample from an easy “envelope” distribution and probabilistically accept/reject to correct the shape.
 $F \rightarrow \tilde{F}$
4. **Ratio-of-Uniforms Method** – generate a pair of uniforms over a cleverly chosen 2-D region whose ratio has the target distribution.
 $\Phi \rightarrow \tilde{\Phi}$

Why so many methods?

No single method is best for every distribution: some have closed-form inverse CDFs (easy), some don't (e.g. Normal), some are only known up to a normalizing constant (common in Bayesian statistics – foreshadowing Module 5!).

Outline

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Theory: The Probability Integral Transform

Algorithm and Continuous Examples

The Discrete Case

Limitations

Acceptance–Rejection Method

Box–Muller Transformation

Ratio-of-Uniforms Method

Diagnosing Randomness

Simulation Error

The Key Theorem

Theorem (Probability Integral Transform) $\rightarrow$ (PIT)

Let F be a continuous, strictly increasing CDF, and let $U \sim \text{Uniform}(0, 1)$. Define

$$X = F^{-1}(U).$$

Then X has CDF F .

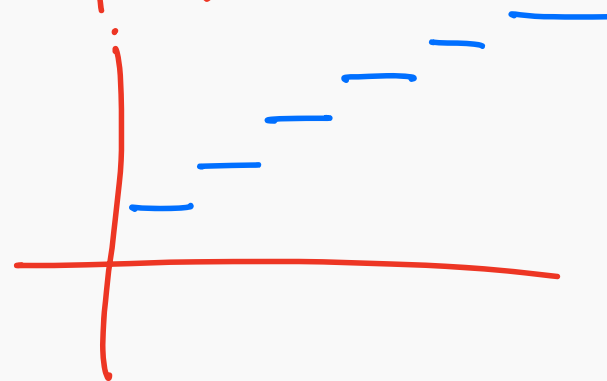
$\downarrow$ discrete
cont's $\rightarrow X$

$$F(x) = P(X \leq x)$$

X : cont



X : disc



$$X = \tilde{F}^{-1}(U), \quad U \sim U[0,1]$$

claim: CDF of X is F

$$\begin{aligned} F_X(x) &= P[X \leq x] \\ &= P[\tilde{F}^{-1}(U) \leq x] \\ &= P\left[U \overset{\text{str}}{\leq} \tilde{F}(x)\right] \end{aligned}$$

$$\Rightarrow U \sim U\left(\begin{smallmatrix} 0,1 \\ a,b \end{smallmatrix}\right)$$

$$F_U(u) = \begin{cases} 0 & u < 0 \\ u & 0 \leq u < 1 \\ 1 & u \geq 1 \end{cases}$$

$$F_X(x) = F(x)$$

$\uparrow$
 CDF of X is

$\rightarrow U_1 = 0.1, U_2 = 0.2$
 $X_1 = \tilde{F}^{-1}(0.1)$
 $X_2 = \tilde{F}^{-1}(0.2) \dots$
 $\vdots$
 $X_n = \dots$

Random number from dist F

The Key Theorem

Theorem (Probability Integral Transform)

Let F be a continuous, strictly increasing CDF, and let $U \sim \text{Uniform}(0, 1)$. Define

$$X = F^{-1}(U).$$

Then X has CDF F .

Proof

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) \\ &= P(U \leq F(x)) \quad (\text{since } F \text{ is monotonic increasing}) \\ &= F(x) \quad (\text{since } U \sim \text{Unif}(0, 1), \text{ so } P(U \leq u) = u). \end{aligned}$$


The Key Theorem

Theorem (Probability Integral Transform)

Let F be a continuous, strictly increasing CDF, and let $U \sim \text{Uniform}(0, 1)$. Define

$$X = F^{-1}(U).$$

Then X has CDF F .

Proof

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) \\ &= P(U \leq F(x)) \quad (\text{since } F \text{ is monotonic increasing}) \\ &= F(x) \quad (\text{since } U \sim \text{Unif}(0, 1), \text{ so } P(U \leq u) = u). \end{aligned}$$

This is the theoretical engine behind the **inverse transform method**: if we can invert F , we can sample from it.

Inverse Transform Algorithm

Algorithm: Inverse Transform Sampling

1. Generate $U \sim \text{Uniform}(0, 1)$. —

2. Compute and return $X = F^{-1}(U)$. —

→ $u_1, u_2, \dots, u_n$
 $x_1 = F^{-1}(u_1), x_2 = F^{-1}(u_2), \dots, x_n = F^{-1}(u_n)$

↳ PRNG: → from F

⇒ we want to simulate from exponential R.V.

$$X \sim \text{Exp}(\lambda) \rightarrow f_X(x) = \begin{cases} e^{-\lambda x} & 0 < x < \infty \\ 0 & \text{otherwise} \end{cases}$$

$$\begin{aligned} F_X(x) &= \begin{cases} 1 - e^{-\lambda x} & 0 < x < \infty \\ 0 & x \leq 0 \end{cases} \\ \Leftrightarrow & \end{aligned}$$

$$X \sim \text{Exp}(\lambda)$$

$$F_X(x) = 1 - e^{-\lambda x}$$

$\in(0,1)$

$$u = 1 - e^{-\lambda x}$$

$$(1-u) = e^{-\lambda x} \rightarrow \log(1-u) = -\lambda x$$

$$\rightarrow x = -\frac{1}{\lambda} \log(1-u)$$

$$x = \bar{F}^{-1}(u) = -\frac{1}{\lambda} \log(1-u)$$

generate $u_1, u_2, \dots, u_n$

$$\lambda = 2$$

$$\begin{aligned} x_1 &= \bar{F}^{-1}(u_1) = -\frac{1}{2} \log(1-u_1) \checkmark \\ x_2 &= \bar{F}^{-1}(u_2) = -\frac{1}{2} \log(1-u_2) \checkmark \\ &\vdots \\ x_n &= \bar{F}^{-1}(u_n) = -\frac{1}{2} \log(1-u_n) \checkmark \end{aligned}$$

PRNG from $\text{exp}(\lambda=2)$

Inverse Transform Algorithm

Algorithm: Inverse Transform Sampling

1. Generate $U \sim \text{Uniform}(0, 1)$.
2. Compute and return $X = F^{-1}(U)$.

Example 1: Exponential Distribution

$F(x) = 1 - e^{-\lambda x}$, $x \geq 0$. Solve $u = 1 - e^{-\lambda x}$ for x :

$$X = F^{-1}(U) = -\frac{\ln(1 - U)}{\lambda}.$$

$\rightarrow$ $U \sim U(0,1)$
 $1-U \sim U(0,1)$

Since $U \sim \text{Unif}(0, 1) \Rightarrow 1 - U \sim \text{Unif}(0, 1)$ too, we may equivalently use

$$X = -\frac{\ln U}{\lambda}.$$

More Continuous Examples

Example 2: Standard Cauchy Distribution ✓

$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$. Inverting:

$$X = \tan\left(\pi\left(U - \frac{1}{2}\right)\right).$$

↓
 $u = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$

$$\pi\left(u - \frac{1}{2}\right) = \arctan(x) \Rightarrow x = \tan\left(\pi\left(u - \frac{1}{2}\right)\right)$$

generate $u_1, u_2, \dots, u_n \sim U(0,1)$ ✓

$$x_1 = \tan\left(\pi\left(u_1 - \frac{1}{2}\right)\right)$$

$$x_n = \tan\left(\pi\left(u_n - \frac{1}{2}\right)\right)$$

Standard
Cauchy

More Continuous Examples

Example 2: Standard Cauchy Distribution

$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$. Inverting:

$$X = \tan\left(\pi\left(U - \frac{1}{2}\right)\right).$$

Example 3: Weibull Distribution

$F(x) = 1 - e^{-(x/\beta)^\alpha}$. Inverting:

$\downarrow$
 $u = 1 - e^{-(x/\beta)^\alpha}$

$\nearrow$
 $\underbrace{X = \beta(-\ln(1 - U))^{1/\alpha}}_{\nearrow}$

More Continuous Examples

Example 2: Standard Cauchy Distribution

$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$. Inverting:

$$X = \tan\left(\pi\left(U - \frac{1}{2}\right)\right).$$

Example 3: Weibull Distribution

$F(x) = 1 - e^{-(x/\beta)^\alpha}$. Inverting:

$$X = \beta \left(-\ln(1 - U) \right)^{1/\alpha}.$$


Example 4: Pareto Distribution

$F(x) = 1 - (x_m/x)^\alpha$, $x \geq x_m$. Inverting:

$$X = \frac{x_m}{(1 - U)^{1/\alpha}}.$$

Inverse Transform for Discrete Random Variables

For a discrete r.v. taking values $x_1 < x_2 < \dots$ with $P(X = x_i) = p_i$, define the CDF as a step function. The algorithm partitions $(0, 1)$ into consecutive intervals of length $p_1, p_2, \dots$.

x_i : discrete.

$x_1, x_2, \dots, x_n$

$P(X = x_i) = p_i, i=1, \dots, n \rightarrow$ pmf of RV X

$x_1 < x_2 < \dots < x_n$

$p_i \geq 0$

$\sum p_i = 1$

$F_X(x) = \begin{cases} 0 & x < x_1 \\ p_1 & x_1 \leq x < x_2 \\ p_1 + p_2 & x_2 \leq x < x_3 \\ \vdots & \vdots \\ \sum_{i=1}^n p_i = 1 & x_n \leq x < x_{n+1} \end{cases}$

$\Leftrightarrow$

$\Rightarrow$ cdf '(s.) is given

Inverse Transform for Discrete Random Variables

For a discrete r.v. taking values $x_1 < x_2 < \dots$ with $P(X = x_i) = p_i$, define the CDF as a step function. The algorithm partitions $(0, 1)$ into consecutive intervals of length $p_1, p_2, \dots$.

Algorithm

1. Generate $U \sim \text{Unif}(0, 1)$.
2. Find the smallest i such that $U \leq \sum_{j \leq i} p_j = F(x_i)$.
3. Return $X = x_i$.

Inverse Transform for Discrete Random Variables

For a discrete r.v. taking values $x_1 < x_2 < \dots$ with $P(X = x_i) = p_i$, define the CDF as a step function. The algorithm partitions $(0, 1)$ into consecutive intervals of length $p_1, p_2, \dots$.

Algorithm

1. Generate $U \sim \text{Unif}(0, 1)$.
2. Find the smallest i such that $U \leq \sum_{j \leq i} p_j = F(x_i)$.
3. Return $X = x_i$.

Example: Biased Coin / Categorical Variable

Suppose $\underline{P(X=1)=0.2}$, $P(X=2)=0.5$, $P(X=3)=0.3$. Then 

$$\begin{aligned} x_1=1 &\rightarrow 0.2 \\ x_2=2 &\rightarrow 0.5 \\ x_3=3 &\rightarrow 0.3 \end{aligned}$$

$$X = \begin{cases} 1 & \text{if } U \in [0, 0.2) \\ 2 & \text{if } U \in [0.2, 0.7) \\ 3 & \text{if } U \in [0.7, 1) \end{cases}$$

A single uniform draw and a lookup table generates the categorical variable.

$$X \Rightarrow 1, 2, 3$$

$$p(X=1) = 0.2$$

$$p(X=2) = 0.5$$

$$p(X=3) = 0.3$$

$$\rightarrow F_X(x) = \begin{cases} 0 & x < 1 \\ 0.2 & 1 \leq x < 2 \\ 0.2 + 0.5 = 0.7 & 2 \leq x < 3 \\ 0.2 + 0.5 + 0.3 = 1 & x \geq 3 \end{cases}$$

$$\Rightarrow U_1, \dots, U_n \sim U[0, 1]$$

$$\Rightarrow U_1 \in (0, 0.2] \rightarrow X_1 = 1, \quad U_1 \in (0.2, 0.7] \rightarrow X_1 = 2, \quad U_1 \in (0.7, 1) \rightarrow X_1 = 3$$

$$\Rightarrow U_1 = 0.8, U_2 = 0.4, U_3 = 0.15, U_4 = 0.6, U_5 = 0.69, U_6 = 0.88$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$X_1 = 2, \quad X_2 = 2, \quad X_3 = 1, \quad X_4 = 2, \quad X_5 = 2, \quad X_6 = 3$$

When the Inverse Transform Method Struggles

- Requires F^{-1} in closed form – not always available.

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- Classic failure case: the Normal distribution. Its CDF $\Phi(x)$ has no elementary closed-form inverse.

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).
 - Workaround 2: use a specialized transformation method instead → Box–Muller (coming up).

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).
 - Workaround 2: use a specialized transformation method instead → **Box–Muller** (coming up).
- For multivariate distributions, F^{-1} is rarely tractable at all.

PIT X

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).
 - Workaround 2: use a specialized transformation method instead → **Box–Muller** (coming up).
- For multivariate distributions, F^{-1} is rarely tractable at all.
- Even when F^{-1} exists, it can be numerically unstable near $U \approx 0$ or $U \approx 1$ (extreme tails).

$$-\log(1-u) \xrightarrow{u \approx 1} \text{Exp}$$

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).
 - Workaround 2: use a specialized transformation method instead → **Box–Muller** (coming up).
- For multivariate distributions, F^{-1} is rarely tractable at all.
- Even when F^{-1} exists, it can be numerically unstable near $U \approx 0$ or $U \approx 1$ (extreme tails).

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).
 - Workaround 2: use a specialized transformation method instead → **Box–Muller** (coming up).
- For multivariate distributions, F^{-1} is rarely tractable at all.
- Even when F^{-1} exists, it can be numerically unstable near $U \approx 0$ or $U \approx 1$ (extreme tails).

Motivates the next methods

When F^{-1} is unavailable or expensive, we turn to acceptance–rejection, special transformations (Box–Muller), and the ratio-of-uniforms method.